

Postman API Testing

- **Authentication APIs**

1. Admin Login
2. User Login

- **User Management APIs (Admin)**

3. Create User
4. Get All Users
5. Search User
6. Update User
7. Delete User

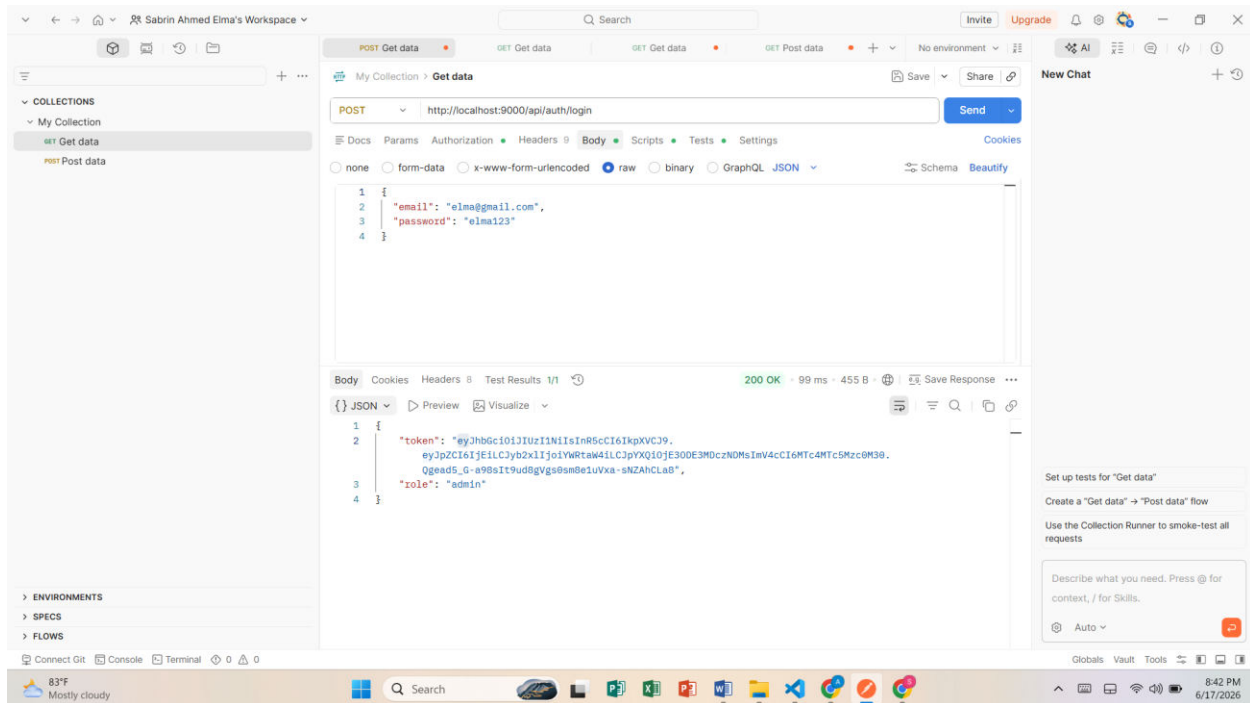
- **Task Management APIs (Admin)**

8. Assign Task
9. Get All Tasks
10. Search Task
11. Update Task
12. Delete Task

- **User Task APIs**

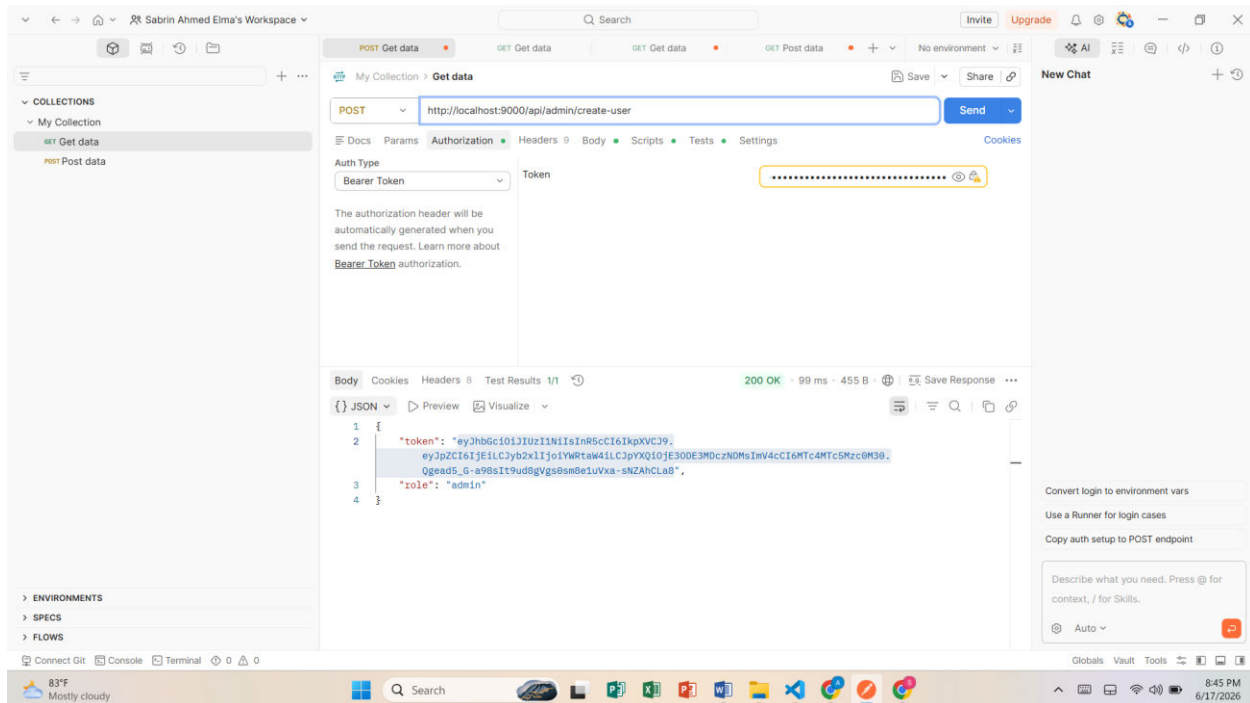
13. View My Tasks
14. Complete Task

1. Admin Login



USER MANAGEMENT

2. Create User



Sabrin Ahmed Elma's Workspace

POST Get data

My Collection > Get data

POST http://localhost:9000/api/admin/create-user

Body

```
1 {
2   "id": "1003",
3   "name": "Israt Jahan",
4   "email": "ij@gmail.com",
5   "password": "123"
6 }
```

201 Created · 109 ms · 391 B

Body

```
1 {
2   "message": "User Created Successfully",
3   "user": {
4     "id": "1003",
5     "name": "Israt Jahan",
6     "email": "ij@gmail.com",
7     "role": "user"
8   }
9 }
```

83°F Mostly cloudy

8:47 PM 6/17/2026

3. Get All Users (READ)

Sabrin Ahmed Elma's Workspace

GET Get data

My Collection > Get data

GET http://localhost:9000/api/admin/users

Auth Type: Bearer Token

Token: [REDACTED]

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

200 OK · 11 ms · 854 B

Body

```
18 {
19   "id": "1001",
20   "name": "Rowshan Nahar",
21   "email": "sam@gmail.com",
22   "password": "$2b$10$ogpfsb6wMiwPyXcMAGtelCGHL7CqUSCGUqcvLN7c2ircMcDcsg1w",
23   "role": "user"
24 },
25 {
26   "id": "1002",
27   "name": "jannatul feirdous ",
28   "email": "jf@gmail.com",
29   "password": "$2b$10$0RAzKV6Zn5XgS1KeheaY3enqKPxIQ017UF6cCt8KHk21UR4JFL3u",
30   "role": "user"
31 },
32 {
33   "id": "1003",
34   "name": "Israt Jahan",
35   "email": "ij@gmail.com",
36   "password": "123",
37   "role": "user"
38 }
39 }
```

83°F Mostly cloudy

8:50 PM 6/17/2026

4. Search User

The screenshot shows the Postman interface with a workspace named "Sabrin Ahmed Elma's Workspace". On the left, the "COLLECTIONS" panel shows "My Collection" with two items: "GET Get data" and "POST Post data". The main panel displays a GET request to `http://localhost:9000/api/admin/search-user?query=israt`. The "Auth Type" is set to "Bearer Token", and a token is entered in the "Token" field. The "Body" tab is selected, showing a JSON response:

```
1 {
2   {
3     "id": "1993",
4     "name": "Israt Jahan",
5     "email": "isj@gmail.com",
6     "password": "$2b$10$7vqKuQPP/BcfZ27vID18S0AGmq4mhYIMxv213veEA.HnDcT05Jfb2",
7     "role": "user"
8   }
9 }
```

The response status is "200 OK" with a response time of 7 ms and a size of 415 B. On the right, the "New Chat" panel is visible with a "Describe what you need. Press @ for context, / for Skills." prompt.

5. Update User

The screenshot shows the Postman interface with the same workspace. The main panel displays a PUT request to `http://localhost:9000/api/admin/update-user/1001`. The "Body" tab is selected, showing a JSON request:

```
1 {
2   "name": "Rowshan Nahar Samiha",
3   "email": "sam@gmail.com"
4 }
```

The response status is "200 OK" with a response time of 11 ms and a size of 468 B. The "Body" tab shows a JSON response:

```
1 {
2   "message": "User Updated Successfully",
3   "user": {
4     "id": 1001,
5     "name": "Rowshan Nahar Samiha",
6     "email": "sam@gmail.com",
7     "password": "$2b$10$10gpf8bWniwPyYxCHAgte1GHL7CqUSCGUqcvLN7c2rcMcDCsg1w",
8     "role": "user"
9   }
10 }
```

On the right, the "New Chat" panel is visible with a "Describe what you need. Press @ for context, / for Skills." prompt.

6. Delete User

The screenshot shows the Postman interface with a workspace named "Sabrin Ahmed Elma's Workspace". The main panel displays a DELETE request to the endpoint `http://localhost:9000/api/admin/delete-user/1002`. The request is configured with a Bearer Token authorization. The response is a 200 OK status with a JSON body:

```
{ 1: { 2: "message": "User Deleted Successfully" 3: }
```

. The left sidebar shows a collection named "My Collection" with a "GET Get data" request. The bottom status bar indicates the system temperature is 83°F and the date is 6/17/2026.

TASK MANAGEMENT

7. Assign Task

The screenshot shows the Postman interface with a workspace named "Sabrin Ahmed Elma's Workspace". The main panel displays a POST request to the endpoint `http://localhost:9000/api/admin/assign-task`. The request is configured with a raw body containing a JSON object:

```
{ 1: { 2: "taskId": "104", 3: "title": "React Dashboard", 4: "description": "Create dashboard UI", 5: "userId": "1003" 6: }
```

. The response is a 201 Created status with a JSON body:

```
{ 1: { 2: "message": "Task Assigned Successfully", 3: "task": { 4: "taskId": "104", 5: "title": "React Dashboard", 6: "description": "Create dashboard UI", 7: "userId": "1003", 8: "status": "Pending" 9: } 10: }
```

. The left sidebar shows a collection named "My Collection" with a "GET Get data" request. The bottom status bar indicates the system temperature is 83°F and the date is 6/17/2026.

8. Get All Tasks

The screenshot shows the Postman interface with a GET request to `http://localhost:9000/api/admin/tasks`. The request is using Bearer Token authentication. The response is a 200 OK status with a response time of 6 ms and a body size of 819 B. The response body is a JSON array of tasks.

taskid	title	description	userid	status	
0	101	Hostel Management System	Design login page UI	1001	Completed
1	102	Inventory Check	Review inventory records and report missing items	1002	Completed
2	103	Customer Feedback Analysis	Analyze customer feedback and prepare improvement suggestions	1002	Pending
3	104	React Dashboard	Create dashboard UI	1003	Pending

9. Search Task

The screenshot shows the Postman interface with a GET request to `http://localhost:9000/api/admin/search-task?query=React`. The request is using Bearer Token authentication. The response is a 200 OK status with a response time of 6 ms and a body size of 383 B. The response body is a JSON array of tasks.

taskid	title	description	userid	status	
0	104	React Dashboard	Create dashboard UI	1003	Pending

10. Update Task

The screenshot shows the Postman interface for a PUT request. The URL is `http://localhost:9000/api/admin/update-task/104`. The request body is a JSON object:

```
1 {
2   "title": "React Dashboard Updated",
3   "description": "Update UI Design",
4   "userId": "1003"
5 }
```

The response is a 200 OK status with a response time of 10 ms and a body of 420 B. The response body is a JSON object:

```
1 {
2   "message": "Task Updated",
3   "task": {
4     "taskId": "104",
5     "title": "React Dashboard Updated",
6     "description": "Update UI Design",
7     "userId": "1003",
8     "status": "Pending"
9   }
10 }
```

11. Delete Task

The screenshot shows the Postman interface for a DELETE request. The URL is `http://localhost:9000/api/admin/delete-task/101`. The request is configured with Bearer Token authentication. The response is a 200 OK status with a response time of 8 ms and a body of 293 B. The response body is a JSON object:

```
1 {
2   "message": "Task Deleted"
3 }
```

The screenshot displays the Postman application interface. On the left sidebar, the 'My Collection' is expanded, showing a 'GET Get data' request. The main workspace shows a 'POST' request to 'http://localhost:9000/ap/auth/login'. The request body is a JSON object with 'email' and 'password' fields. The response is a JSON object with 'token', 'role', and 'user' fields. The status bar at the bottom indicates a successful response (200 OK) with a response time of 95 ms and a body size of 454 B.

Request Details:

- Method: POST
- URL: http://localhost:9000/ap/auth/login
- Body (raw):


```
{
  "email": "sam@gmail.com",
  "password": "123"
}
```

Response Details:

- Status: 200 OK
- Response Time: 95 ms
- Body (raw):


```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImFtPSa1cm95ZS16InVzZXIjLCJpYXQiOiJlODU0e3M0KHNzQsInV4cCI6MTc4MTc5NTg3NjBh",
  "role": "user"
}
```

The screenshot displays the Postman application interface. On the left sidebar, the 'My Collection' is expanded, showing a 'GET Get data' request. The main workspace shows the 'GET Get data' request editor. The URL is 'http://localhost:9000/ap/user/my-tasks'. The 'Auth Type' is set to 'Bearer Token'. The 'Token' field is highlighted with a yellow box. The 'Body' tab is selected, showing a '200 OK' status and a JSON response. The JSON response is an array of tasks, with the first task highlighted in blue.

taskid	title	description	userid	status
0106	Social Media Management	Schedule and publish content on company social media accounts	1001	Pending

14. User Complete Task

The screenshot displays a REST client interface within a workspace named "Sabrin Ahmed Elma's Workspace". The interface is divided into several sections:

- Left Sidebar:** Contains a "COLLECTIONS" section with "My Collection" expanded, showing two items: "GET Get data" and "POST Post data". Below this are sections for "ENVIRONMENTS", "SPECS", and "FLOWS".
- Top Bar:** Includes a search bar, "Invite", "Upgrade", and other utility icons.
- Main Panel:** Shows a PUT request to the endpoint `http://localhost:9000/api/user/complete-task/106`. The request body is a JSON object:

```
{ 1: { 2: "email": "sam@gmail.com", 3: "password": "123" 4: }
```

. The response is a 200 OK status with a response time of 10 ms and a body of

```
{ 1: { 2: "message": "Task Completed" 3: }
```

.
- Bottom Bar:** Displays system information such as "83°F Mostly cloudy", a search bar, and various application icons. The system clock shows "9:27 PM 6/17/2026".